

TRABAJO PRÁCTICO N° 3

Seminario de Práctica de Informática

Sistema de Gestión Integral para una Clínica de Salud

Alumna: Antonella Diaz

DNI: 28.910.424

Comisión: VINFOR12606

Índice

1. Explicación del Desarrollo en Java

1.1 Estructura general del sistema

1.2 Conceptos de POO aplicados

1.3 Diagrama de Clases UML

2. Presentación del Desarrollo en Java

2.1 Main.java

2.2 SistemaGestionClinica.java

2.3 Persona.java (clase abstracta)

2.4 Paciente.java

2.5 Medico.java

2.6 Cita.java

2.7 Producto.java e Inventario.java

2.8 Factura.java y FacturaDAO.java

2.9 Patrón DAO y ConexionDB

3. Compilación y Ejecución del Programa

1. Explicación del Desarrollo en Java

1.1 Estructura General del Sistema

El sistema fue desarrollado aplicando los principios de la Programación Orientada a Objetos (POO) con el objetivo de construir un software modular, mantenible y extensible. La decisión de separar cada entidad del dominio —pacientes, médicos, citas, inventarios y facturas— en clases propias responde a la necesidad de que cada componente tenga una única responsabilidad bien definida, facilitando tanto su comprensión como su modificación futura sin afectar al resto del sistema.

La arquitectura final se organiza en tres capas: la capa de presentación (menú de consola en SistemaGestionClinica), la capa de lógica de negocio (clases de dominio como Paciente, Medico, Cita, Producto y Factura) y la capa de acceso a datos (patrón DAO con conexión a MySQL mediante JDBC). Esta separación permite, por ejemplo, reemplazar la base de datos sin necesidad de modificar la lógica de negocio.

1.2 Conceptos de POO Aplicados

Encapsulamiento

Todos los atributos de las clases de dominio se declararon como privados (private). El acceso y la modificación de estos datos se realizan exclusivamente a través de métodos públicos (getters y setters). Esta decisión protege la integridad del objeto: por ejemplo, no es posible asignar directamente un valor inválido al campo "estado" de una Cita sin pasar por el método correspondiente, donde se podría agregar validación en el futuro.

Herencia

Se definió la clase abstracta Persona como base para Paciente y Medico. Esta decisión evita duplicar atributos comunes (nombre, apellido, dirección, teléfono, correo) en ambas clases. Al heredar de Persona, tanto Paciente como Medico incorporan automáticamente estos atributos y sus métodos de acceso, y sólo agregan lo que les es propio: la fecha de nacimiento en Paciente y la especialidad en Medico. Esto reduce significativamente el código repetido y mantiene una jerarquía clara.

Polimorfismo

El método toString() fue sobrescrito (override) en cada clase para devolver una representación legible del objeto. Esto permite mostrar información de cualquier entidad de forma uniforme, independientemente de si es un Paciente o un Medico. El mismo mecanismo se aplica en el patrón DAO: la interfaz DAO<T> define operaciones genéricas que cada DAO concreto implementa según las particularidades de su tabla. El sistema puede llamar a insertar() o obtenerTodos() sin conocer si está operando sobre pacientes, médicos o facturas.

Abstracción

Se utilizó la abstracción en dos niveles: la clase abstracta Persona, que define la estructura común pero delega los detalles a sus subclasses, y la interfaz DAO<T>, que establece el

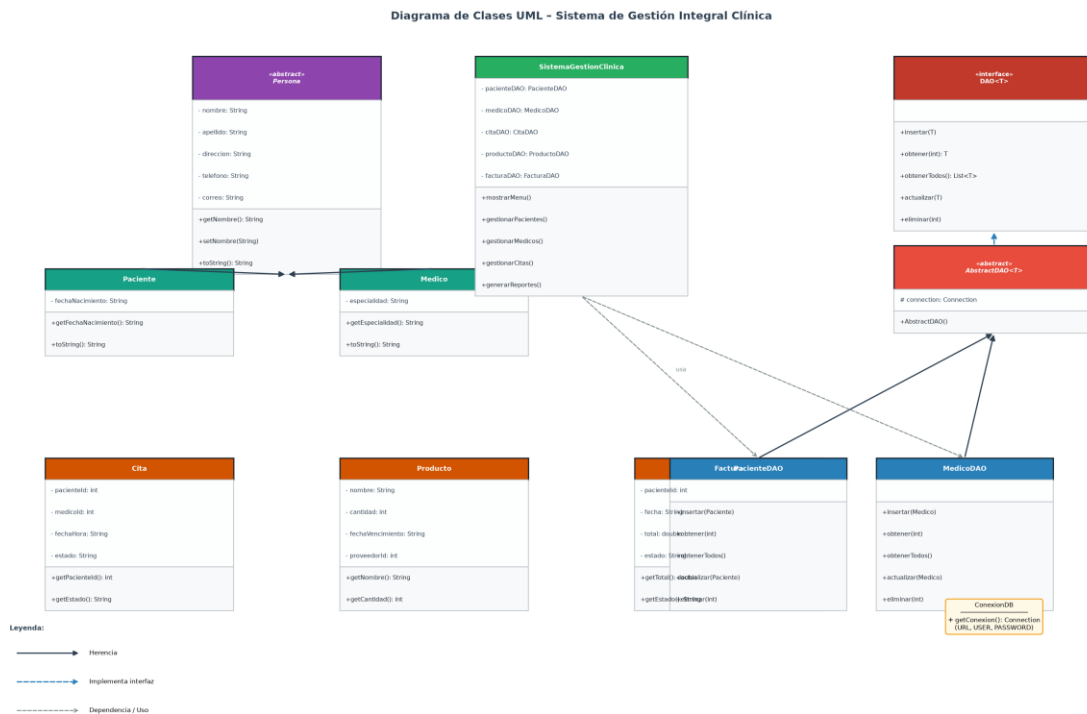
contrato de las operaciones de persistencia sin definir su implementación. Esta capa de abstracción permite que el resto del sistema dependa de contratos (interfaces) y no de implementaciones concretas, un principio clave del diseño orientado a objetos conocido como "programar hacia la interfaz, no hacia la implementación".

Manejo de Excepciones

Las operaciones de base de datos pueden fallar por múltiples razones: conexión perdida, violación de clave foránea, duplicados, etc. Para manejar estos escenarios sin que el programa se detenga abruptamente, se utilizó el manejo de excepciones con bloques try-catch en todos los métodos de los DAOs. Las excepciones de tipo SQLException se capturan y se muestra un mensaje descriptivo al usuario, permitiendo que el sistema continúe operando ante un error puntual.

1.3 Diagrama de Clases UML

El siguiente diagrama muestra la estructura de clases del sistema, sus atributos, métodos y las relaciones entre ellas. Las flechas con triángulo vacío representan herencia, las flechas punteadas con triángulo indican implementación de interfaz, y las flechas punteadas simples representan relaciones de uso o dependencia.



2. Presentación del Desarrollo en Java

2.1 Main.java

Esta clase contiene el método `main()`, que es el punto de entrada del programa. Su función es instanciar el sistema de gestión y delegar el control al menú principal. Se decidió mantener esta clase lo más simple posible, siguiendo el principio de responsabilidad única: Main sólo inicia la aplicación, sin contener lógica de negocio.

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        SistemaGestionClinica sistema = new SistemaGestionClinica();
        sistema.mostrarMenu();
    }
}
```

2.2 SistemaGestionClinica.java

Es la clase principal de la aplicación. Centraliza el menú de opciones y coordina el acceso a cada módulo del sistema. En lugar de implementar la lógica directamente, delega las operaciones a los DAOs correspondientes, actuando como un controlador que conecta la interfaz de usuario con la capa de datos. Inicializa una instancia de cada DAO al arrancar, lo que abre la conexión con la base de datos de forma anticipada y la mantiene disponible durante toda la sesión.

```
public class SistemaGestionClinica {
    private PacienteDAO pacienteDAO = new PacienteDAO();
    private MedicoDAO medicoDAO = new MedicoDAO();
    private CitaDAO citaDAO = new CitaDAO();
    private ProductoDAO productoDAO = new ProductoDAO();
    private FacturaDAO facturaDAO = new FacturaDAO();
    private Inventario inventario = new Inventario();

    public void mostrarMenu() {
        Scanner scanner = new Scanner(System.in);
        int opcion;
        do {
            System.out.println("=== Sistema de Gestion Integral ===");
            System.out.println("1. Gestion de Pacientes");
            System.out.println("2. Gestion de Medicos");
            System.out.println("3. Gestion de Citas");
            System.out.println("4. Historias Clinicas");
            System.out.println("5. Gestion de Inventarios");
            System.out.println("6. Facturacion y Pagos");
            System.out.println("7. Reportes de Gestion");
            System.out.println("8. Salir");
            System.out.print("Seleccione una opcion: ");
            opcion = scanner.nextInt();
            // ... switch con llamadas a métodos privados por módulo
        } while (opcion != 8);
    }
}
```

2.3 Persona.java (Clase Abstracta)

Persona es la clase base de la jerarquía de herencia. Se definió como abstracta porque no tiene sentido instanciar una "Persona" genérica en el contexto de la clínica: siempre se trabaja con Pacientes o Médicos específicos. Sin embargo, ambos comparten los mismos atributos de identificación personal, por lo que centralizar estos datos en Persona elimina duplicación y facilita el mantenimiento: si en el futuro se agrega un atributo (como número de documento), se modifica en un único lugar y ambas subclases lo heredan automáticamente.

```
public abstract class Persona {
    private int    id;
    private String nombre;
    private String apellido;
    private String direccion;
    private String telefono;
    private String correo;

    // Constructor, getters y setters
    public String getNombre() { return nombre; }
    public String getApellido(){ return apellido; }
    // ... resto de métodos de acceso
}
```

2.4 Paciente.java

Paciente extiende Persona y agrega el atributo fechaNacimiento, que es específico de los pacientes y no aplica a los médicos. Gracias a la herencia, un objeto Paciente tiene disponibles todos los métodos de Persona sin necesidad de redefinirlos. El constructor llama a `super()` para inicializar los atributos heredados y luego asigna el propio.

```
public class Paciente extends Persona {
    private String fechaNacimiento;

    public Paciente(String nombre, String apellido, String fechaNacimiento,
                    String direccion, String telefono, String correo) {
        super();
        setNombre(nombre); setApellido(apellido);
        setDireccion(direccion); setTelefono(telefono); setCorreo(correo);
        this.fechaNacimiento = fechaNacimiento;
    }
    public String getFechaNacimiento() { return fechaNacimiento; }
}
```

2.5 Medico.java

De forma análoga a Paciente, Medico hereda de Persona y agrega el atributo especialidad. Esta diferenciación permite que el sistema trate a médicos y pacientes de forma polimórfica cuando sea necesario (por ejemplo, para mostrar listas combinadas) y de forma específica cuando se requieren sus atributos propios.

```
public class Medico extends Persona {
    private String especialidad;

    public Medico(String nombre, String apellido, String especialidad,
```

```

        String telefono, String correo) {
    super();
    setNombre(nombre); setApellido(apellido);
    setTelefono(telefono); setCorreo(correo);
    this.especialidad = especialidad;
}
public String getEspecialidad() { return especialidad; }
}

```

2.6 Cita.java

Cita representa un turno médico y relaciona a un paciente con un médico en una fecha y hora determinada. En lugar de almacenar los objetos completos de Paciente y Medico, se optó por guardar únicamente sus IDs (claves foráneas), siguiendo el mismo modelo que la base de datos. Esto reduce el uso de memoria y evita inconsistencias entre el objeto en memoria y los datos persistidos.

```

public class Cita {
    private int id;
    private int pacienteId;
    private int medicoId;
    private String fechaHora;
    private String estado;

    public Cita(int pacienteId, int medicoId, String fechaHora, String estado)
    {
        this.pacienteId = pacienteId;
        this.medicoId = medicoId;
        this.fechaHora = fechaHora;
        this.estado = estado;
    }
    // Getters y setters
}

```

2.7 Producto.java e Inventario.java

Producto almacena la información de cada ítem del inventario: nombre, cantidad, fecha de vencimiento y proveedor. La clase Inventario actúa como contenedor de productos y ofrece operaciones de alto nivel como agregar, eliminar, buscar y mostrar. Esta separación respeta el principio de responsabilidad única: Producto sabe qué es un producto, e Inventario sabe cómo gestionar una colección de ellos. Adicionalmente, la clase EstadisticasInventario calcula el promedio de unidades disponibles usando un array de Producto[], demostrando el uso de arrays en el contexto real.

```

public class Inventario {
    private ArrayList<Producto> productos = new ArrayList<>();

    public void agregarProducto(Producto p) { productos.add(p); }
    public void eliminarProducto(Producto p) { productos.remove(p); }
    public Producto buscarProducto(String nombre) {
        for (Producto p : productos)
            if (p.getNombre().equalsIgnoreCase(nombre)) return p;
        return null;
    }
    public void mostrarInventario() {

```

```

        for (Producto p : productos)
            System.out.printf("%d | %s | %d | %s\n",
                               p.getId(), p.getNombre(), p.getCantidad(),
                               p.getFechaVencimiento());
    }
}

```

2.8 Factura.java

Factura representa un comprobante de pago vinculado a un paciente. El atributo estado permite distinguir entre facturas pendientes de cobro y facturas ya pagadas, lo que resulta esencial para el módulo de reportes, que calcula el total de ingresos sumando únicamente las facturas con estado "Pagada". El total se define como double para soportar valores con decimales (centavos).

```

public class Factura {
    private int id;
    private int pacienteId;
    private String fecha;
    private double total;
    private String estado;

    public Factura(int pacienteId, String fecha, double total, String estado) {
        this.pacienteId = pacienteId;
        this.fecha = fecha;
        this.total = total;
        this.estado = estado;
    }

    public double getTotal() { return total; }
    public String getEstado() { return estado; }
}

```

2.9 Patrón DAO y ConexionDB

El patrón DAO (Data Access Object) fue elegido como estrategia de acceso a datos porque permite separar completamente la lógica de negocio de las operaciones SQL. La interfaz DAO<T> define el contrato genérico con cinco operaciones CRUD. La clase abstracta AbstractDAO<T> implementa esta interfaz y establece la conexión a la base de datos en su constructor, compartiéndola con todas las subclases. Cada DAO concreto (PacienteDAO, MedicoDAO, CitaDAO, ProductoDAO, FacturaDAO) hereda de AbstractDAO y provee la implementación SQL específica para su tabla.

```

public interface DAO<T> {
    void insertar(T t) throws SQLException;
    T obtener(int id) throws SQLException;
    List<T> obtenerTodos() throws SQLException;
    void actualizar(T t) throws SQLException;
    void eliminar(int id) throws SQLException;
}

public abstract class AbstractDAO<T> implements DAO<T> {
    protected Connection connection;
    public AbstractDAO() {
        try {
            this.connection = ConexionDB.getConexion();
        }
    }
}

```



```

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}

```

La clase `ConexionDB` centraliza los parámetros de conexión (URL, usuario, contraseña) en un único lugar. Si en el futuro el sistema migra a otro servidor o motor de base de datos, sólo se modifica esta clase, sin tocar ningún DAO:

```

public class ConexionDB {
    private static final String URL =

    "jdbc:mysql://localhost:3306/clinicadb?useSSL=false&serverTimezone=UTC";
    private static final String USER    = "root";
    private static final String PASSWORD = "123456";

    public static Connection getConexion() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASSWORD);
    }
}

```

Ejemplo de DAO concreto — `PacienteDAO.insertar()` usa un `PreparedStatement` para evitar inyección SQL y asigna los parámetros con sus tipos correctos:

```

public void insertar(Paciente paciente) {
    String query = "INSERT INTO pacientes "
        + "(nombre, apellido, fecha_nacimiento, direccion, telefono, "
        + "correo) "
        + "VALUES (?, ?, ?, ?, ?, ?)";
    try (PreparedStatement ps = connection.prepareStatement(query)) {
        ps.setString(1, paciente.getNombre());
        ps.setString(2, paciente.getApellido());
        ps.setString(3, paciente.getFechaNacimiento());
        ps.setString(4, paciente.getDireccion());
        ps.setString(5, paciente.getTelefono());
        ps.setString(6, paciente.getCorreo());
        ps.executeUpdate();
    } catch (SQLException e) {
        System.err.println("Error al insertar paciente: " + e.getMessage());
    }
}

```

3. Compilación y Ejecución del Programa

Para compilar y ejecutar el sistema es necesario contar con el JDK instalado y el driver JDBC de MySQL (mysql-connector.jar) en la misma carpeta que los archivos .java. El sistema fue probado con JDK 21 y MySQL Connector/J 9.1.0.

Requisitos previos

- JDK 21 o superior instalado
- MySQL Server en ejecución con la base de datos ClinicaDB creada
- Archivo mysql-connector.jar en la misma carpeta que los .java

Compilación

Desde la terminal, en la carpeta que contiene los archivos Java:

```
javac -cp ".;mysql-connector.jar" *.java
# En Linux/Mac usar ":" en lugar de ";":
javac -cp ".:mysql-connector.jar" *.java
```

El flag -cp especifica el classpath: incluye el directorio actual (.) y el driver JDBC. Compilar todos los archivos juntos (*.java) asegura que las dependencias entre clases se resuelvan correctamente.

Ejecución

```
java -cp ".;mysql-connector.jar" Main
```

Al ejecutarse, el sistema establece la conexión con MySQL y presenta el menú principal. Desde allí el usuario puede gestionar pacientes, médicos, citas, historias clínicas, inventarios, facturación y generar reportes de gestión, con todos los datos persistidos en la base de datos ClinicaDB.

Resultado esperado

```
=== Sistema de Gestion Integral para Clinicas de Salud ===
1. Gestion de Pacientes
2. Gestion de Medicos
3. Gestion de Citas
4. Historias Clinicas
5. Gestion de Inventarios
6. Facturacion y Pagos
7. Reportes de Gestion
8. Salir
Seleccione una opcion: _
```

Cada opción despliega un submenú que permite ver los registros existentes o ingresar nuevos datos. Los reportes muestran estadísticas en tiempo real: total de pacientes, médicos, citas pendientes, promedio de stock en inventario e ingresos totales por facturas pagadas.